

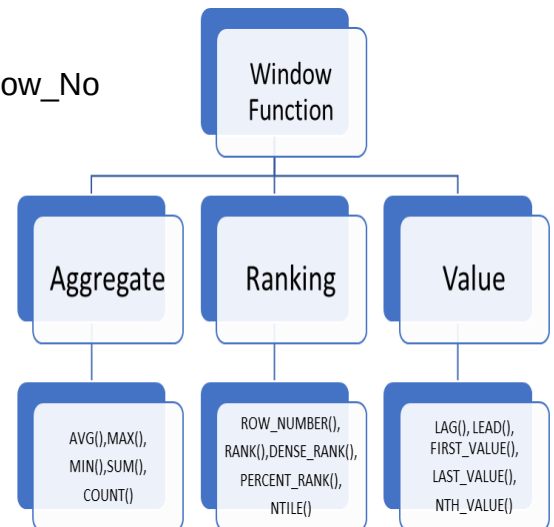
1.Window Function:-

1. ROW_NUMBER():

Assigns a unique number to each row.

```
SELECT EmpID, EmpName,DeptID,Salary,  
ROW_NUMBER() OVER(ORDER BY Salary DESC) AS Row_No  
FROM Employee;
```

Result Grid				Filter Rows:	
	EmpID	EmpName	Salary	RoW_Num	
▶	3	Sakshi	40000.00	1	
	2	Priyank	35000.00	2	
	5	Neha	35000.00	3	
	10	Raj	32000.00	4	
	1	Rutuja	30000.00	5	
	9	Prajakta	30000.00	6	
	7	Rutuja	25000.00	7	
	4	Akanksha	22000.00	8	
	8	Vipul	20000.00	9	



```
SELECT EmpID, EmpName, Salary,DeptId,  
ROW_NUMBER() OVER(partition by DeptId Order BY Salary DESC) AS RowNum  
FROM Employee;
```

EmpID	EmpName	Salary	DeptId	RowNum
1	Rutuja	30000.00	101	1
9	Prajakta	30000.00	101	2
3	Sakshi	40000.00	102	1
2	Priyank	35000.00	102	2
5	Neha	35000.00	102	3
10	Raj	32000.00	102	4
7	Rutuja	25000.00	103	1
4	Akanksha	22000.00	103	2
8	Vipul	20000.00	104	1
6	Amruta	18000.00	104	2

2. RANK():

Employees with the same salary get the same rank, and the next rank is skipped

```
SELECT EmpID,EmpName,Salary,  
RANK() OVER(ORDER BY Salary DESC) AS Rank_No FROM Employee;
```

	EmpID	EmpName	Salary	Rank_No
▶	3	Sakshi	40000.00	1
	2	Priyank	35000.00	2
	5	Neha	35000.00	2
	10	Raj	32000.00	4
	1	Rutuja	30000.00	5
	9	Prajakta	30000.00	5
	7	Rutuja	25000.00	7
	4	Akanksha	22000.00	8
	8	Vipul	20000.00	9
	6	Amruta	18000.00	10

3. DENSE_RANK():

Employees with the same salary get the same rank, but no ranks are skipped.

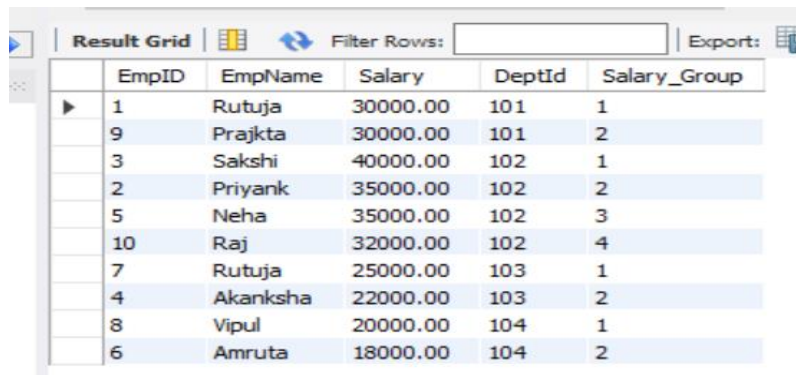
```
SELECT EmpID, EmpName, Salary,DeptName,  
dense_rank() OVER( order by Salary DESC) AS SalaryRank FROM Employee e  
INNER JOIN Department d ON e.DeptID = d.DeptID;
```

	EmpID	EmpName	Salary	DeptName	SalaryRank
▶	3	Sakshi	40000.00	IT	1
	2	Priyank	35000.00	IT	2
	5	Neha	35000.00	IT	2
	10	Raj	32000.00	IT	3
	1	Rutuja	30000.00	HR	4
	9	Prajakta	30000.00	HR	4
	7	Rutuja	25000.00	Marketing	5
	4	Akanksha	22000.00	Marketing	6
	8	Vipul	20000.00	Sales	7
	6	Amruta	18000.00	Sales	8

4. NTILE():

Divides rows into equal groups.

```
SELECT EmpID,EmpName,Salary,DeptId,  
NTILE(4) OVER(Partition By DeptId ORDER BY Salary DESC) AS Salary_Group  
FROM Employee;
```



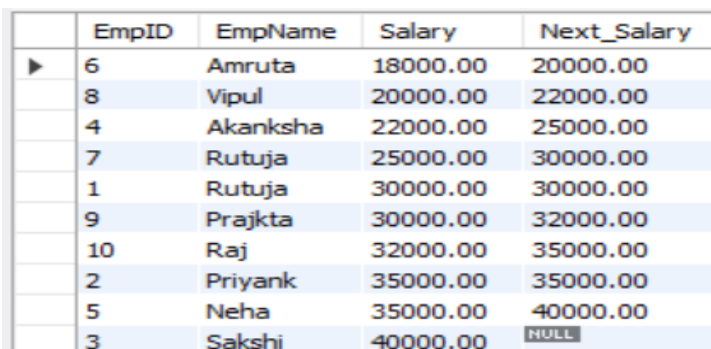
The screenshot shows a 'Result Grid' with columns: EmpID, EmpName, Salary, DeptId, and Salary_Group. The data is sorted by DeptId and then by Salary in descending order. The Salary_Group column shows the result of the NTILE(4) function, which divides the rows into 4 groups based on salary within each department.

	EmpID	EmpName	Salary	DeptId	Salary_Group
▶	1	Rutuja	30000.00	101	1
	9	Prajakta	30000.00	101	2
	3	Sakshi	40000.00	102	1
	2	Priyank	35000.00	102	2
	5	Neha	35000.00	102	3
	10	Raj	32000.00	102	4
	7	Rutuja	25000.00	103	1
	4	Akanksha	22000.00	103	2
	8	Vipul	20000.00	104	1
	6	Amruta	18000.00	104	2

6. LEAD():

Shows the next row's value.

```
SELECT EmpID,EmpName,Salary,  
LEAD(Salary) OVER(ORDER BY Salary) AS Next_Salary  
FROM Employee;
```



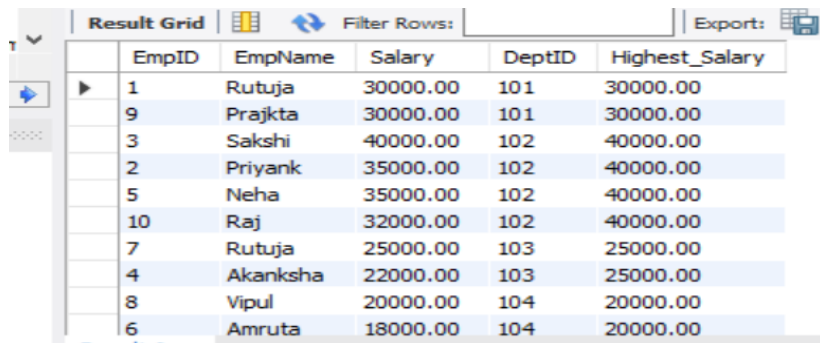
The screenshot shows a table with columns: EmpID, EmpName, Salary, and Next_Salary. The data is sorted by Salary in descending order. The Next_Salary column shows the salary of the next row in the sorted order. The last row, for EmpID 3 (Sakshi), has a NULL value in the Next_Salary column.

	EmpID	EmpName	Salary	Next_Salary
▶	6	Amruta	18000.00	20000.00
	8	Vipul	20000.00	22000.00
	4	Akanksha	22000.00	25000.00
	7	Rutuja	25000.00	30000.00
	1	Rutuja	30000.00	30000.00
	9	Prajakta	30000.00	32000.00
	10	Raj	32000.00	35000.00
	2	Priyank	35000.00	35000.00
	5	Neha	35000.00	40000.00
	3	Sakshi	40000.00	NULL

7. FIRST_VALUE():

Returns the first salary in the ordered window.

```
SELECT EmpID,EmpName, Salary,DeptID,  
  
FIRST_VALUE(Salary) OVER(partition by DeptID order by Salary DESC) AS  
Highest_Salary  
  
FROM Employee ;
```

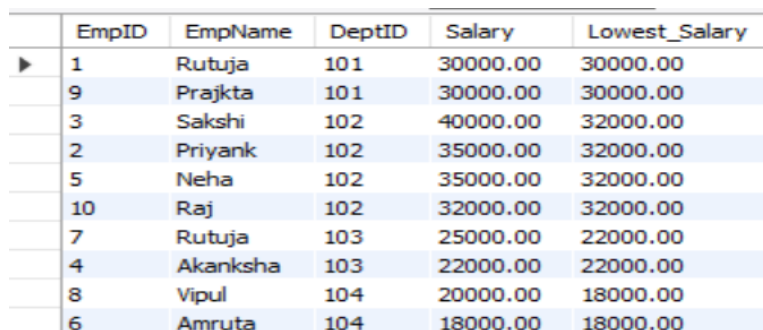


	EmpID	EmpName	Salary	DeptID	Highest_Salary
1	1	Rutuja	30000.00	101	30000.00
9	9	Prajakta	30000.00	101	30000.00
3	3	Sakshi	40000.00	102	40000.00
2	2	Priyank	35000.00	102	40000.00
5	5	Neha	35000.00	102	40000.00
10	10	Raj	32000.00	102	40000.00
7	7	Rutuja	25000.00	103	25000.00
4	4	Akanksha	22000.00	103	25000.00
8	8	Vipul	20000.00	104	20000.00
6	6	Amruta	18000.00	104	20000.00

8. LAST_VALUE()

Returns the last value in the window.

```
SELECT EmpID, EmpName,DeptID, Salary,  
  
LAST_VALUE(Salary) OVER (partition by DeptID order by Salary DESC  
  
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
  
) AS Lowest_Salary  
  
FROM Employee;
```



	EmpID	EmpName	DeptID	Salary	Lowest_Salary
1	1	Rutuja	101	30000.00	30000.00
9	9	Prajakta	101	30000.00	30000.00
3	3	Sakshi	102	40000.00	32000.00
2	2	Priyank	102	35000.00	32000.00
5	5	Neha	102	35000.00	32000.00
10	10	Raj	102	32000.00	32000.00
7	7	Rutuja	103	25000.00	22000.00
4	4	Akanksha	103	22000.00	22000.00
8	8	Vipul	104	20000.00	18000.00
6	6	Amruta	104	18000.00	18000.00

2.Trigger:

```
use empdb;
```

```
CREATE TABLE Employee_Log
```

```
(
```

```
LogID INT AUTO_INCREMENT PRIMARY KEY,
```

```
EmpID INT,
```

```
EmpName VARCHAR(50),
```

```
ActionPerformed VARCHAR(20),
```

```
ActionTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

```
DELIMITER //
```

```
CREATE TRIGGER after_insert_tri
```

```
AFTER INSERT
```

```
ON Employee_log_3
```

```
FOR EACH ROW
```

```
BEGIN
```

```
INSERT INTO Employee_Log(EmpID, EmpName, ActionPerformed)
```

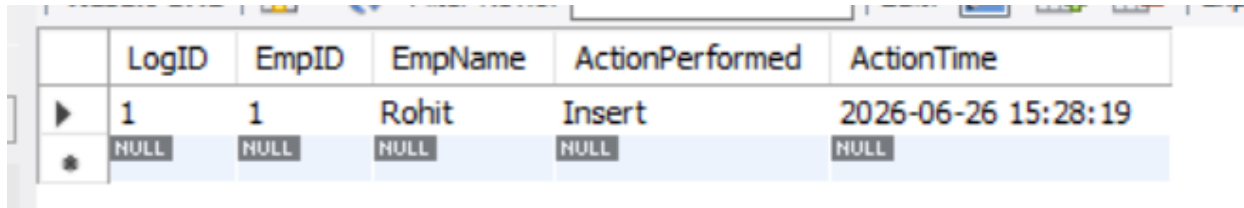
```
VALUES (NEW.EmpID, NEW.EmpName, 'INSERT');
```

```
END //
```

```
DELIMITER ;
```

```
INSERT INTO Employee_log (EmpID, EmpName,Action Performed)VALUES  
(1, 'Rohit','Insert');
```

```
SELECT * FROM Employee_Log;
```



	LogID	EmpID	EmpName	ActionPerformed	ActionTime
▶	1	1	Rohit	Insert	2026-06-26 15:28:19
*	NULL	NULL	NULL	NULL	NULL

Update

```
DELIMITER //
```

```
CREATE TRIGGER trg_employeeelog_update
```

```
AFTER UPDATE ON Employee
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO Employee_Log (EmpID, EmpName, ActionPerformed)
```

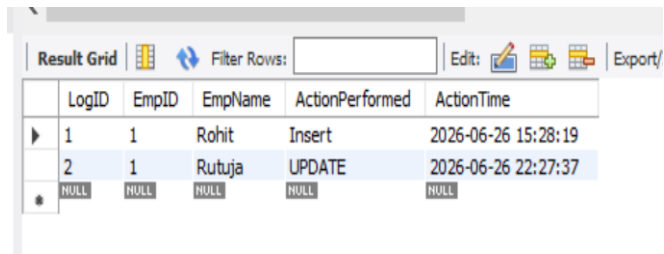
```
    VALUES (NEW.EmpID, NEW.EmpName, 'UPDATE');
```

```
END //
```

```
DELIMITER ;
```

UPDATE Employee SET EmpName = "Rutuja"

WHERE EmpID = 1;



A screenshot of a database application's 'Result Grid'. The grid has columns: LogID, EmpID, EmpName, ActionPerformed, and ActionTime. It contains three rows: Row 1 (LogID: 1, EmpID: 1, EmpName: Rohit, ActionPerformed: Insert, ActionTime: 2026-06-26 15:28:19), Row 2 (LogID: 2, EmpID: 1, EmpName: Rutuja, ActionPerformed: UPDATE, ActionTime: 2026-06-26 22:27:37), and Row 3 (all fields are NULL). Above the grid is a toolbar with 'Result Grid', 'Filter Rows' (with a search box), 'Edit', and 'Export' buttons.

LogID	EmpID	EmpName	ActionPerformed	ActionTime
1	1	Rohit	Insert	2026-06-26 15:28:19
2	1	Rutuja	UPDATE	2026-06-26 22:27:37
	NULL	NULL	NULL	NULL

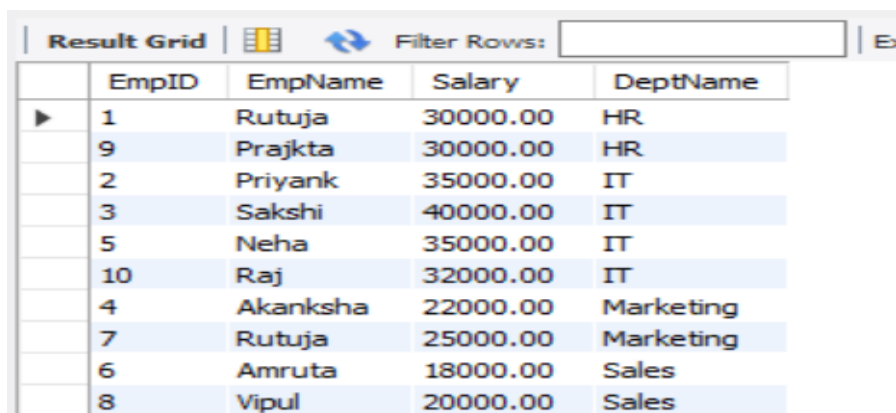
3.View:

SELECT e.EmpID,e.EmpName,e.Salary, d.DeptName FROM Employee e

JOIN Department d

ON e.DeptID=d.DeptID;

SELECT * FROM EmployeeDepartmentView1;



A screenshot of a database application's 'Result Grid' showing the output of a SELECT query. The grid has columns: EmpID, EmpName, Salary, and DeptName. It contains 10 rows of employee data. Above the grid is a toolbar with 'Result Grid', 'Filter Rows' (with a search box), and 'Export' buttons.

EmpID	EmpName	Salary	DeptName
1	Rutuja	30000.00	HR
9	Prajakta	30000.00	HR
2	Priyank	35000.00	IT
3	Sakshi	40000.00	IT
5	Neha	35000.00	IT
10	Raj	32000.00	IT
4	Akanksha	22000.00	Marketing
7	Rutuja	25000.00	Marketing
6	Amruta	18000.00	Sales
8	Vipul	20000.00	Sales

4: case when

```
SELECT EmpID,EmpName,Salary,CASE  
WHEN Salary>=40000 THEN 'High Salary'  
WHEN Salary>=25000 THEN 'Medium Salary'  
ELSE 'Low Salary'  
END AS SalaryCategory  
FROM Employee;
```

Result Grid					Filter Rows:	Export
	EmpID	EmpName	Salary	SalaryCategory		
▶	1	Rutuja	30000.00	Medium Salary		
	2	Priyank	35000.00	Medium Salary		
	3	Sakshi	40000.00	High Salary		
	4	Akanksha	22000.00	Low Salary		
	5	Neha	35000.00	Medium Salary		
	6	Amruta	18000.00	Low Salary		
	7	Rutuja	25000.00	Medium Salary		
	8	Vipul	20000.00	Low Salary		
	9	Prajakta	30000.00	Medium Salary		
	10	Raj	32000.00	Medium Salary		